

SISTEMAS DISTRIBUÍDOS TALLER 7

PROYECTO FINAL - PARTE IV

OBJETIVO: Desplegar la cuarta y última parte de la base del proyecto final de curso, evaluando conocimientos técnicos en manejo de sesiones, HTTP y WS.

Mira tu sistema actual y responde honestamente:

- Coordinadores: Tienes varios. Si uno se cae, los clientes migran. Es decir, es tolerante a fallos.
- Cliente: Cada navegador es independiente. No es un punto de falla.
- Auth: Es un solo proceso.

Hoy, si matamos el auth, pasan dos cosas:

1. Ningún cliente nuevo puede entrar. No hay login, no hay `/auth/google`, no hay registro.
2. Los coordinadores no pueden registrar nuevos heartbeats y los clientes existentes no pueden pedir un coordinador nuevo cuando el suyo se cae.

Es decir: Los que ya están jugando siguen jugando (porque el JWT se valida sin consultar al auth), pero el sistema queda congelado: nadie nuevo entra, nadie se reconecta. Es un sistema "casi distribuido" donde el corazón sigue siendo único.

Este taller arregla eso. Vamos a tener múltiples auth-services replicados, y el sistema sobrevive a la muerte de cualquiera de ellos.

Los coordinadores eran fáciles de replicar porque su estado era derivable. Todos pueden calcular las mismas posiciones a partir de los mismos intents. El auth es distinto: su estado es la tabla de usuarios, y las escrituras (registrar usuario, vincular Google) son operaciones que no se pueden derivar. Si dos auth-services aceptan registrar `alice` al mismo tiempo, ¿quién gana?, ¿qué password queda? Y peor: ¿qué `userId` tiene? Porque otros servicios firman JWTs con ese `userId`.

Hay muchas formas de resolverlo. En este taller vamos a utilizar un concepto simple que funciona:

- Un líder a la vez. Solo el líder acepta escrituras. Los demás son réplicas pasivas (read-only).
- Las réplicas leen del líder y mantienen su propia copia local de la BD.
- Si el líder muere, los demás detectan que falta y eligen uno nuevo entre ellos.

- Los clientes y coordinadores se conectan a cualquier auth. Si pegan a una réplica que recibió una escritura, la réplica la redirige al líder.

Esto se llama single-writer replication en la industria. Es lo que hacían MySQL clásico, PostgreSQL primary-replica, MongoDB antes de los config servers. No es lo más sofisticado, pero es lo que sostiene buena parte de internet y es completamente alcanzable con lo que ya hemos trabajado.

1. Roles

Todo auth-service puede estar en uno de dos estados:

leader

- Acepta lecturas y escrituras (`/register`, `/login`, `/auth/google`).
- Cada escritura se aplica a su BD local y se propaga a los **replicas** por WS.
- Emite heartbeats a los **replicas** cada 2 segundos.

replica

- Acepta sólo lecturas (`/login` requiere consultar la BD para verificar password; eso es lectura).
- Rechaza escrituras con `503 { error: "not_leader", leaderUrl: "..." }`. El cliente puede reintentar contra el líder o el coordinador puede usar la URL del leader directamente.
- Recibe cambios del líder y los aplica a su BD local.
- Si no recibe heartbeats del líder por 6 segundos (3 ciclos), considera que el líder murió e inicia una elección.

Importante: Un replica NO emite tokens si su BD está desactualizada respecto al líder.

2. Lo qué se va a construir

Paso 1: Endpoints nuevos (HTTP)

GET /status

Response:

```
JSON
{
  "authId": "auth-a",
  "role": "leader",
  "publicUrl": "http://localhost:4001",
  "peerUrl": "ws://localhost:4101",
  "leaderUrl": "http://localhost:4001",
  "knownPeers": ["auth-b", "auth-c"],
  "lastAppliedSeq": 142,
  "users": 27
}
```

- **role**: "leader" o "replica".
- **leaderUrl**: Cuál es el líder actual desde la perspectiva de este nodo. Si eres el líder, eres tú mismo.
- **lastAppliedSeq**: Número de secuencia del último cambio aplicado. Sirve para detectar si una réplica está atrasada.
- **users**: Cuántos usuarios tiene en su BD local. Útil para debug y demo (cuando haces un **register**, el contador sube en todos).

GET /peers

Devuelve la lista de auth-services conocidos por este nodo (igual concepto que en el Taller anterior con coordinadores).

Response:

```
JSON
{
  "peers": [
    { "authId": "auth-a", "publicUrl": "http://...", "peerUrl":
      "ws://...", "role": "leader" },
    { "authId": "auth-c", "publicUrl": "http://...", "peerUrl":
      "ws://...", "role": "replica" }
  ]
}
```

Paso 2: Endpoints existentes que cambian de comportamiento

POST /register, POST /login, POST /auth/google:

- Si eres **leader**: comportamiento normal.
- Si eres **replica**:
 - **Para escrituras** (register, primera vez de Google): Devuelves **503 { error: "not_leader", leaderUrl: "http://localhost:4001" }**. El cliente o el coordinador hace el request contra **leaderUrl**.
 - **Para lecturas** (login, Google de usuario existente): Puedes responder si tu **lastAppliedSeq** está cerca del del líder. Si estás muy atrasado, también devuelve **503 not_leader**. (Pista: durante una elección no sabés quién es el líder; devuelve **503 { error: "no_leader" }**)

Paso 3: Protocolo del mesh de auth

Cada auth se conecta a todos los demás por WS en un puerto separado. La resolución del "quién inicia" sigue el mismo patrón: el de authId lexicográficamente menor inicia.

Handshake (primer mensaje al abrir la conexión):

```
JSON
{ "type": "hello", "authId": "auth-a", "role": "leader", "term": 3 }
```

Heartbeat (emitido por un líder cada 2 segundos a todas las réplicas):

JSON

```
{ "type": "heartbeat", "authId": "auth-a", "term": 3, "lastSeq": 142 }
```

Propagar (emitido por el líder cuando aplica una escritura):

JSON

```
{
  "type": "write_propagate",
  "term": 3,
  "seq": 143,
  "op": "register",
  "data": {
    "userId": 28,
    "username": "alice",
    "provider": "local",
    "password_hash": "$2b$10$...",
    "created_at": "2026-..."
  }
}
```

Tipos de **op**: "register", "register_google", etc. (uno por cada tipo de escritura).

Sincronizar petición (emitido por una réplica al líder cuando su **lastSeq** está atrasado):

JSON

```
{ "type": "request_sync", "fromSeq": 100 }
```

Sincronizar respuesta (emitido por el líder con todas las escrituras desde **fromSeq**):

JSON

```
{
  "type": "sync_response",
  "entries": [
    { "seq": 101, "op": "register", "data": {...} },
    { "seq": 102, "op": "register_google", "data": {...} },
    ...
  ]
}
```

Mensajes para elección de líder

Election (emitido por una réplica que detecta que el líder murió):

JSON

```
{ "type": "election", "candidate": "auth-b", "term": 4, "lastSeq": 142 }
```

Vote (respuesta a una elección):

JSON

```
{ "type": "vote", "voter": "auth-c", "term": 4, "voteGranted": true }
```

New leader (anuncio cuando alguien ganó la elección):

JSON

```
{ "type": "new_leader", "leader": "auth-b", "term": 4 }
```

Paso 4: Consistency model

Cuándo se considera "aplicada" una escritura en el líder

Opción A — Aplicación inmediata (más rápida, menos segura): El líder aplica a su BD y responde 200 al cliente. Después propaga a las réplicas en background.

Riesgo: Si el líder muere justo después de responder pero antes de propagar, ese cambio se pierde. Algún cliente cree que se registró pero el sistema no lo tiene.

Opción B — Propagación primero, respuesta después (más lenta, más segura): el líder propaga a las réplicas, espera confirmación de al menos una (o de la mayoría), y solo entonces responde al cliente.

Riesgo: Si una réplica es lenta, todas las escrituras se ralentizan. Si dos réplicas están caídas, las escrituras se bloquean.

Tienes que elegir una y justificarla en tu README.

Qué pasa con lecturas en réplicas atrasadas

Una réplica con `lastAppliedSeq = 100` cuando el líder está en `seq = 142` está 42 escrituras atrasada. Si un cliente le pide `/login`, ¿le responde?

Dos opciones:

- **Sí, porque para esa réplica el usuario probablemente exista** (la mayoría de los usuarios ya están en `seq < 100`).
- **No, redirigir al líder**, porque hay un usuario nuevo (`seq = 138`) que la réplica todavía no conoce y le va a decir "credenciales inválidas" cuando en realidad las credenciales son válidas.

Decide y justifica. Como recomendación: Si `leaderSeq - mySeq < 10` (tolerancia configurable), responde localmente. Si está más atrás, redirigir al líder con `503 not_leader`.

Qué pasa durante una elección

Mientras no haya un líder, ninguna escritura se acepta. Es por unos segundos. Documenta que las escrituras pueden fallar durante una elección y que el cliente debe reintentar.

Paso 5: Cambios en el resto del sistema

Cliente

- En `config.js` (o en el `.env`), en vez de un solo `AUTH_URL`, ahora hay un array `AUTH_URLS`: `['http://localhost:4001', 'http://localhost:4002', 'http://localhost:4003']`.
- Cuando hace cualquier request a auth, prueba con el primero. Si recibe error de red o `5xx`, prueba con el siguiente. Si recibe `503 not_leader`, hace el request directamente contra `leaderUrl`.
- Muestra en algún lado de la UI a qué auth está conectado.

Coordinadores

- Mismo cambio: `AUTH_URLS` como lista, lógica de fallback igual.
- Los heartbeats al directorio los manda al líder. Si el líder cambia, se reorienta.

3. Entrega y sustentación

Qué se entrega:

1. **Link al repo de GitHub**. Commits incrementales, no uno solo.
2. **README completo**.
3. **Sistema corriendo con túneles en ngrok durante la sustentación**.

Qué se sustenta:

1. Levantás 3 auths y 3 coordinadores. Muestras el `/status` de cada auth: uno es leader, dos son replicas. Los tres tienen `users: N` igual. (3 min).
2. Registras un usuario `alice` en el sistema. Muestras que después del request, los 3 auths tienen `users: N+1`. Es decir: la escritura se propagó. (3 min).
3. Matas al líder con Ctrl+C. Esperas ~10 segundos. Haces `GET /status` a las dos réplicas vivas. Una de las dos ahora dice `role: "leader"`. Esto demuestra que hubo elección. (2 min).
4. Registras un usuario `bob` después de la elección. Funciona. El nuevo líder lo acepta y propaga. (4 min).
5. Levantas el auth muerto. Cuando vuelve, detecta que su `lastSeq` está atrasado y pide `sync`. Después del sync, su contador `users` está al día. Haces `GET /status` y muestra `role: "replica"` (no recupera el liderazgo, se incorpora como réplica). (3 min).

Mientras todo esto pasa, los clientes que estaban jugando NO se desconectan. Esto es porque el coordinador valida JWT sin auth. Demuéstralo: deja un cliente jugando durante toda la sesión de demo. Que el círculo siga moviéndose mientras matás auths y se hacen elecciones.